

Cluster algorithms for Monte Carlo simulation

Part II Computing Project

April 2024

Abstract

We implement the Metropolis-Hastings algorithm for the Ising model and a hard-disk fluid model, and compare the performance to cluster algorithms by making measurements of the autocorrelation time τ_c and the runtime/complexity of the algorithms. For the Ising model, we find that the cluster algorithm is more efficient than the Metropolis-Hastings algorithm, particularly near the critical point, but that it does not perform well at small values of a . For the hard-disk model, we find that the Metropolis-Hastings algorithm is more efficient than the cluster algorithm at low number densities, but that for even moderate densities, the cluster algorithm vastly outperforms it.

1 Introduction

Monte Carlo methods are an extremely diverse and powerful set of techniques for solving problems in a wide range of fields. In physics, they are used to simulate the behaviour of complex systems whose partition function is difficult or impossible to evaluate. In particular, they are most commonly used to estimate the equilibrium values of macroscopic quantities of interest. In this paper, we will consider Markov chain Monte Carlo methods, and the so-called *cluster algorithms* which are designed to improve their efficiency. We implement the most famous MCMC algorithm, the Metropolis-Hastings algorithm, for the Ising model and a hard-disk fluid model, and compare the performance to corresponding cluster algorithms for each model.

2 Markov chain Monte Carlo (MCMC)

Before exploring specifics, we review the key properties of Markov chains, and why they are useful in Monte Carlo simulation. Typically, our goal in such a simulation of a physical system is to estimate the expectation value of some quantity:

$$\langle Q \rangle = \frac{\sum_{\mu} Q_{\mu} e^{-\beta E_{\mu}}}{\sum_{\mu} e^{-\beta E_{\mu}}}$$

For all but the smallest systems, we cannot evaluate this for all states. We instead must compute the **estimator** of Q :

$$Q_M := \frac{\sum_{i=1}^M Q_{\mu_i} p_{\mu_i}^{-1} e^{-\beta E_{\mu_i}}}{\sum_{i=1}^M p_{\mu_i}^{-1} e^{-\beta E_{\mu_i}}}$$

where now we are using a finite number, M , of states. We would like to compute the best possible estimate for the least number of samples, so we make use of **importance sampling**, where the probability distribution p_μ is such that we choose with maximal likelihood the states in which the system spends the most time. In physical systems, this amounts to choosing states with a probability distribution $p_\mu = Z^{-1}e^{-\beta E_\mu}$. This gives a more simple expression for the estimator:

$$Q_M = \frac{1}{M} \sum_{i=1}^M Q_{\mu_i}$$

The question is *how* we choose states with this probability distribution. It would be inefficient to choose states uniformly and then accept them according to p_μ . This is where Markov chains become an essential tool.

2.1 Markov chains

A Markov chain is a set of random variables X_i satisfying the **Markov property**:

$$P(X_{i+1} \in A | X_j = x_j, 0 \leq j \leq i) = P(X_{i+1} \in A | X_i = x_i)$$

where A is any subset of the state space of the chain. In other words, the transition from one state to the next is independent of all previous states.

- A Markov chain is **time-homogeneous** iff it satisfies

$$P(X_{i+1} \in A | X_i = x_i) = P(X_1 \in A | X_0 = x_0).$$

- Define the **transition matrix**, having elements $P_{jk} \equiv P(\omega_j \rightarrow \omega_k) := P(X_i = \omega_k | X_{i-1} = \omega_j)$, where ω_i are states in the state space.
- Denote the **probability distribution** as a vector $\mathbf{p}_i = (p_{X_i}(\omega_1), \dots, p_{X_i}(\omega_M))$. Since

$$\begin{aligned} p_{X_{i+1}}(\omega_k) &= P(X_{i+1} = \omega_k) \\ &= \sum_{j=1}^M P(X_i = \omega_j) P(X_{i+1} = \omega_k | X_i = \omega_j) \\ &= \sum_{j=1}^M p_{X_i}(\omega_j) P_{jk} \end{aligned}$$

$\mathbf{p}_i = \mathbf{p}_0 \mathbf{P}^i$ for a time-homogeneous Markov chain.

- Define π to be a **stationary distribution** iff $\pi = \pi \mathbf{P}$ (note that the subscript has been dropped, and that the notation is ‘lazy’ i.e. the vector notation is omitted).
- The transition matrix is **irreducible** iff for every pair of states x, y in the state space, there is a positive probability that a Markov chain started in state x will eventually reach state y . This is crucially linked to the idea of **ergodicity**: the property of a system that, given an infinite amount of time, it will explore its entire state space.

- The transition matrix is **aperiodic** iff $P_{ii} > 0 \forall i$ i.e. a state can always stay the same.

It is a consequence of the Perron-Frobenius theorem [1] that if $\mathbf{P}$ is *irreducible* and *aperiodic*, the Markov chain always converges to a stationary distribution π . This is crucial for Monte Carlo, because we can get a Markov chain to converge to any desired distribution by finding an appropriate $\mathbf{P}$. It turns out that there are many [2]. The stationary distribution condition can be written:

$$\pi = \pi \mathbf{P}$$

$$\sum_x \pi(x) P(x \rightarrow y) = \pi(y) = \pi(y) \sum_x P(y \rightarrow x) = \sum_x \pi(y) P(y \rightarrow x)$$

We guarantee this by enforcing the **detailed balance** condition:

$$\pi(x) P(x \rightarrow y) = \pi(y) P(y \rightarrow x). \quad (1)$$

2.2 Metropolis-Hastings algorithm

The following is a general procedure for constructing a Markov process satisfying (1) [3].

Let $P(x \rightarrow y) = Q(x \rightarrow y)A(x \rightarrow y)$, where Q is a known proposal distribution and A is the acceptance probability. The detailed balance condition can be written:

$$\pi(x) Q(x \rightarrow y) A(x \rightarrow y) = \pi(y) Q(y \rightarrow x) A(y \rightarrow x). \quad (2)$$

Let $A(y \rightarrow x) = s \pi(x) Q(x \rightarrow y)$, $s \geq 0$. Then (2) becomes $A(x \rightarrow y) = s \pi(y) Q(y \rightarrow x)$. Since $A(x \rightarrow y), A(y \rightarrow x) \leq 1$, choose s such that the larger of the two is equal to 1 i.e.

$$s = \frac{1}{\max[\pi(y) Q(y \rightarrow x), \pi(x) Q(x \rightarrow y)]}.$$

This gives acceptance probability

$$A(x \rightarrow y) = \min \left(1, \frac{\pi(y) Q(y \rightarrow x)}{\pi(x) Q(x \rightarrow y)} \right). \quad (3)$$

The Metropolis-Hastings algorithm is then:

1. Choose a proposal distribution $Q(x \rightarrow y)$.
2. Choose an initial state x_0 .
3. For each iteration i :
 - i. Propose a new state y according to $Q(x \rightarrow y)$.
 - ii. Accept y with probability $A(x \rightarrow y)$ given by (3).
 - iii. If accepted, $x_i = y$, otherwise $x_i = x_{i-1}$.

Provided the chain is ergodic, the distribution of states will converge to π . Note: the proposal distribution can be chosen for convenience, we need not know the normalisation constant of π , and the acceptance ratio is maximal, so that the chain is as efficient as possible.

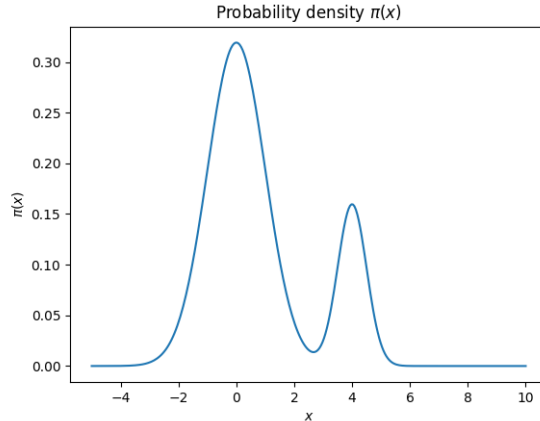


Figure 1: A bimodal distribution

2.3 Autocorrelation

Suppose we have a desired distribution π which is bimodal (see Fig 1), and our Q involves proposing $X_{i+1} = X_i + U$, where U is a random variable chosen from a uniform distribution which is narrower than the peaks of π . Even once the chain has converged, it will take a long time to explore the entire state space, as the chain will tend to stay in one peak for a long time before jumping to the other. This is known as **autocorrelation**, where the process retains some ‘memory’ of previous states.

We can measure this phenomenon: for some quantity q , the **correlation** is [2]

$$\chi(t) = \int dt' [q(t')q(t' + t) - \langle q \rangle^2]$$

or the discrete equivalent:

$$\chi(t) = \frac{1}{n-t} \left[\sum_{t'=1}^{n-t} q(t')q(t' + t) - \langle q \rangle^2 \right]. \quad (4)$$

We assume that for long times t , $\chi(t) \sim e^{-t/\tau_c}$, where τ_c is the **autocorrelation time**. A serious problem for Metropolis-Hastings algorithms is that the autocorrelation time can sometimes become very large, meaning that the chain will take a long time to give good estimators of the quantities of interest: in general we must average over measurements spaced by $2\tau_c$ [2]. One class of algorithms which seek to combat this are known as **cluster algorithms**. There is no general method for producing these, so we must turn to specific examples.

3 The 2D Ising model

First we consider the Ising model, a well-known simple model of a ferromagnet. The Hamiltonian is given by

$$H = -J \sum_{ij,nn} s_i s_j - \mu B \sum_i s_i,$$

where $s_i = \pm 1$. In our implementations, we work with a dimensionless $A \equiv \beta H$, and also take $B = 0$, such that for one spin, we have ‘energy’

$$A_i = -a s_i \sum_{j,nn} s_j,$$

where $a \equiv \beta J$.

Onsager’s solution is

$$\langle M \rangle = \begin{cases} 0 & \text{for } T > T_c \\ (1 - \sinh^{-4}(2\beta J))^{1/8} & \text{for } T < T_c \end{cases} \quad (5)$$

$$= \begin{cases} 0 & \text{for } a < a_c \\ (1 - \sinh^{-4}(2a))^{1/8} & \text{for } a > a_c \end{cases} \quad (6)$$

in terms of our dimensionless parameters. The critical value of a is

$$a_c = \frac{\ln(1 + \sqrt{2})}{2}. \quad (7)$$

3.1 Critical slowing-down

We can also define a spatial correlation between spins:

$$C(\mathbf{r}) = \langle s_{\mathbf{x}} s_{\mathbf{x}+\mathbf{r}} \rangle - \langle s_{\mathbf{x}} \rangle \langle s_{\mathbf{x}+\mathbf{r}} \rangle \sim e^{-|\mathbf{r}|/\xi},$$

where ξ is the correlation length. It is a result from *conformal field theory* that near the critical point, $\xi \sim |T - T_c|^{-\nu}$, where $\nu = 1$ is a critical exponent [4]. The canonical assumption [2] is that $\tau_c \sim \xi^z \sim |T - T_c|^{-z\nu}$, where z is an algorithm-dependent value called the **dynamic exponent**. We can see then that a poor choice of algorithm might give a very large autocorrelation time near the critical point. This phenomenon is called **critical slowing-down**.

3.2 Metropolis-Hastings algorithm for the Ising model

We wish to sample the Boltzmann distribution, so the Metropolis-Hastings algorithm is as follows:

1. Choose a spin uniformly at random and propose flipping it i.e. $s_i \rightarrow -s_i$.
2. Since, from above, our $Q(x \rightarrow y) = Q(y \rightarrow x)$, accept the move with probability

$$\begin{aligned} A(x \rightarrow y) &= \min \left(1, \frac{\pi(y)Q(y \rightarrow x)}{\pi(x)Q(x \rightarrow y)} \right) \\ &= \min \left(1, e^{\beta H(x) - \beta H(y)} \right) \\ &= \min \left(1, e^{-\Delta A} \right). \end{aligned}$$

3. Repeat until each spin has been proposed to flip once.
4. Repeat the above until the chain has converged.

This algorithm suffers from critical slowing-down: near the critical point, the correlation length diverges, so we see the formation of large domains.¹ Destroying these domains is time consuming for the Metropolis-Hastings algorithm, since it considers each spin individually.

¹Indeed, at the critical point, there will be a *percolating domain* spanning even an infinite lattice.

3.2.1 Implementation

We make a few observations. Firstly, $\Delta A = -2A_i$ for a single spin flip. Secondly, since we are using Python, the most significant increase in efficiency will come from vectorization. We can achieve it by using Boolean masking to consider all the spins on a checkerboard pattern at once, since these spins are independent of each other. Finally, based on [2] we use a lookup table for the exponential function. Floating point calculations are fast on modern CPUs, but for large lattice sizes and many iterations, the time adds up. We impose periodic boundary conditions on the lattice. Our full implementation is outlined as follows:

```
checkerboard_1 = array(shape=(N,N), entries=(1 IF i+j even,
                                              0 otherwise))
checkerboard_2 = array(shape=(N,N), entries=(1 IF i+j odd,
                                              0 otherwise))
# Lookup table for exp(-2A) s.t. the neighbour sum is the index
lut = exp([0, 0, -4a, 0, -8a, 8a, 0, 4a, 0])

FOR i IN range(iterations):
    p = array(shape=(N,N), entries=random_uniform(0,1))

    partial_lattice_1 = lattice * checkerboard_1 # Element-wise
    partial_lattice_2 = lattice * checkerboard_2 # Element-wise
    to_flip = array(shape=(N,N),
                    entries=(1 IF p[i,j] <
                                lut[neighbour_sum(lattice)], 0 otherwise))
    partial_lattice_1[to_flip] *= -1
    lattice = partial_lattice_1 + partial_lattice_2 # Element-wise

    partial_lattice_2 = lattice * checkerboard_2 # Element-wise
    to_flip = array(shape=(N,N),
                    entries=(1 IF p[i,j] <
                                lut[neighbour_sum(lattice)], 0 otherwise))
    partial_lattice_2[to_flip] *= -1
    lattice = partial_lattice_1 + partial_lattice_2 # Element-wise
```

Note: firstly, while the checkerboard Boolean masks allow us to flip only the correct spins, the neighbour sum and lookup are performed for the whole lattice. This introduces redundancy into the calculation, but it is a necessary trade-off for the vectorization. We expect the benefit to outweigh the cost. Secondly, we do not evaluate the Metropolis-Hastings acceptance probability in the form (3). Our method is equivalent, but comes with the benefit that we can treat all the spins the same way, rather than separating the favourable and unfavourable moves. Since we are using the lookup table, there is no significant cost to this.

3.2.2 Preliminary testing

We wish to demonstrate that our implementation reproduces the expected behaviour. Firstly, we initialize a 128×128 lattice with all spins pointing in the same direction, and

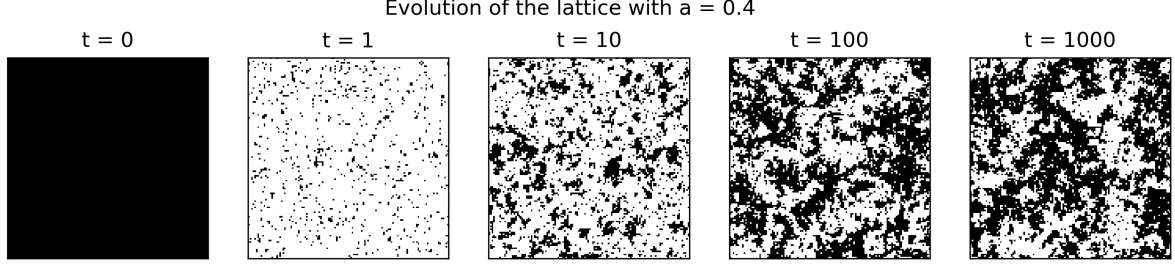


Figure 2: The evolution of a 128×128 lattice using the Metropolis-Hastings algorithm with $a = 0.4$. The lattice is initialised with all spins pointing in the same direction. The lattice is shown at $t = 0, 1, 10, 100, 1000$ iterations. The black squares represent spins pointing up, while the white squares represent spins pointing down.

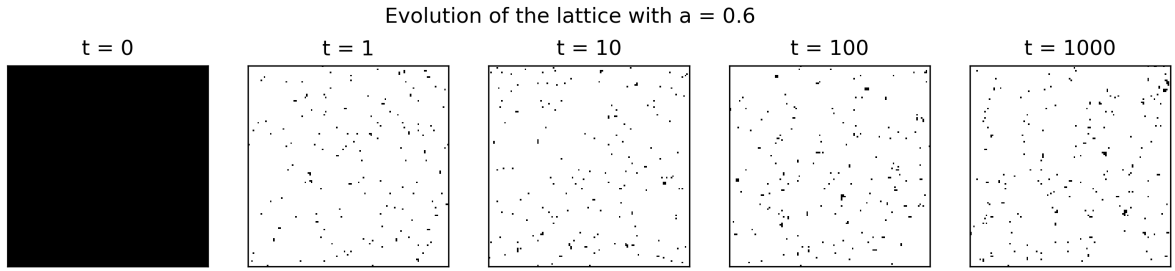


Figure 3: The evolution of a 128×128 lattice using the Metropolis-Hastings algorithm with $a = 0.6$. The lattice is initialised with all spins pointing in the same direction. The lattice is shown at $t = 0, 1, 10, 100, 1000$ iterations. The black squares represent spins pointing up, while the white squares represent spins pointing down.

evolve the system for 1000 iterations, taking samples at $t = 0, 1, 10, 100, 1000$ iterations. Secondly, we run each algorithm for 200 iterations at a range of values of a and compare the absolute relative magnetisation to the Onsager expression (6). The results are shown in Figs 2-4.

We see that the Metropolis-Hastings algorithm converges to the expected behaviour. Recall we do not place requirements on *how* the lattice converges. Note also the fluctuations of the magnetization about the predicted equilibrium value. These are expected in a thermal system, and are seen here because the plotted values are not averages.

3.3 Cluster algorithm for the Ising model - Wolff algorithm

There are multiple cluster algorithms for the Ising model; the most famous is the Wolff algorithm [5]. This is designed to combat critical slowing-down by breaking up large domains. The algorithm is as follows:

1. Choose a random spin s_i as the seed for a cluster. Let $\sigma = s_i$ be the value of this spin.
2. For each neighbouring spin with value σ , add the spin to the cluster with probability $P_{add} = 1 - e^{-2\beta J} = 1 - e^{-2a}$.
3. For each spin added to the cluster, repeat step 2 for only those neighbours that have not already been considered.

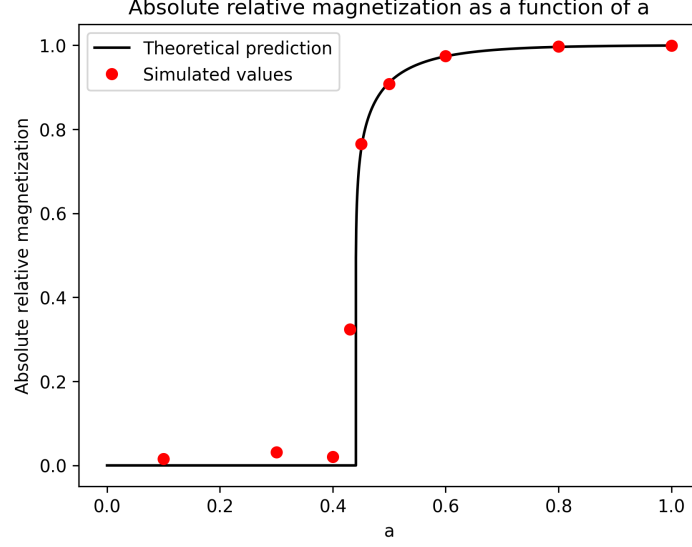


Figure 4: The absolute relative magnetisation of a 128×128 lattice as a function of a using the Metropolis-Hastings algorithm. The positive branch of the Onsager solution is shown for comparison. To produce the data for this plot, an initially aligned lattice was evolved for 200 iterations of the Metropolis-Hastings algorithm at each of 9 values of a . We take the absolute value because on a given run, the lattice randomly aligns preferentially either up or down.

4. Repeat until no new spins are added to the cluster.
5. Flip the entire cluster.

It is slightly non-trivial to see that this algorithm satisfies detailed balance; the reasoning can be found in Appendix A.

3.3.1 Implementation

We implement the Wolff algorithm almost exactly as described, the only modification being that we flip the spins as we go:

```
p_add = 1-exp(-2*a)
```

```
FOR i IN iterations:
```

```
    seed = [randint_uniform(0,N),randint_uniform(0,N)]
```

```
    spin = lattice[seed[0],seed[1]]
```

```
    unvisited = [seed]
```

```
    WHILE len(unvisited) > 0:
```

```
        site = unvisited.pop() # access and remove last element
```

```
        FOR neighbour IN neighbours(site): # get indices of neighbours
```

```
            IF lattice[neighbour[0],neighbour[1]] == spin:
```

```
                p = random_uniform(0,1)
```

```
                IF p < p_add:
```

```
                    lattice[neighbour[0],neighbour[1]] *= -1
```

```
                    unvisited.append_left(neighbour) # add to the front
```

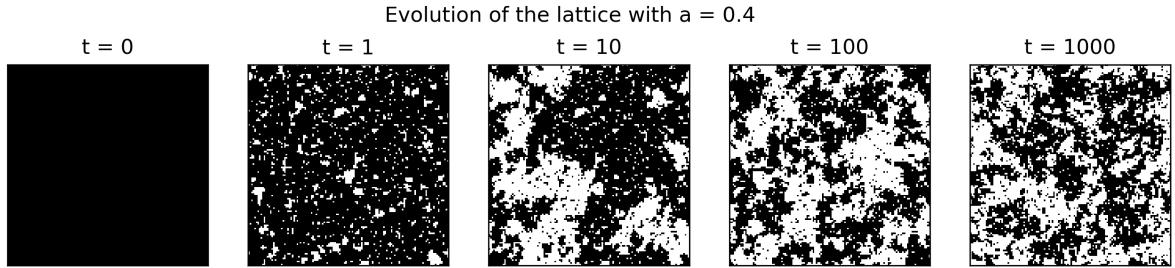


Figure 5: The evolution of a 128×128 lattice using the Wolff algorithm with $a = 0.4$. The lattice is initialised with all spins pointing in the same direction. The lattice is shown at $t = 0, 1, 10, 100, 1000$ iterations. The black squares represent spins pointing up, while the white squares represent spins pointing down.

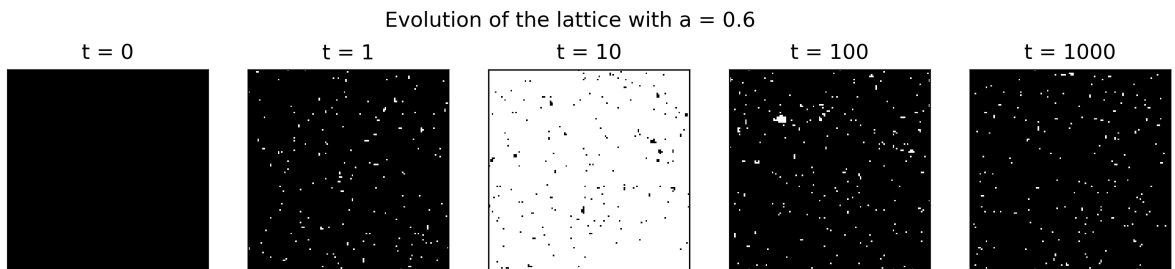


Figure 6: The evolution of a 128×128 lattice using the Wolff algorithm with $a = 0.6$. The lattice is initialised with all spins pointing in the same direction. The lattice is shown at $t = 0, 1, 10, 100, 1000$ iterations. The black squares represent spins pointing up, while the white squares represent spins pointing down.

3.3.2 Preliminary testing

We repeat the preliminary tests described in Section 3.2 for the Wolff algorithm. The results are shown in Figs 5-7.

We can see that the Wolff algorithm also largely reproduces the expected behaviour. The exception is at small values of a , where it appears to over-predict the magnetisation. The likely explanation is that the algorithm takes a long time to converge at these values, since the average cluster size tends toward one spin.

3.4 Comparison

We have seen that both algorithms are viable for simulating the physics; we now investigate the relative efficiency of the two algorithms under different conditions.

3.4.1 Autocorrelation time

We first investigate the autocorrelation time of the algorithms. The method for measuring the autocorrelation time is as follows: we run the algorithm for a large number of iterations, then calculate the correlation of the absolute relative magnetization according to (4), discarding the beginning of the time-series where the system is yet to converge. We then take τ_c as the time at which the correlation falls to $1/e$ of its initial value. This is crude but fast and gives an acceptable estimate. It also avoids the imple-

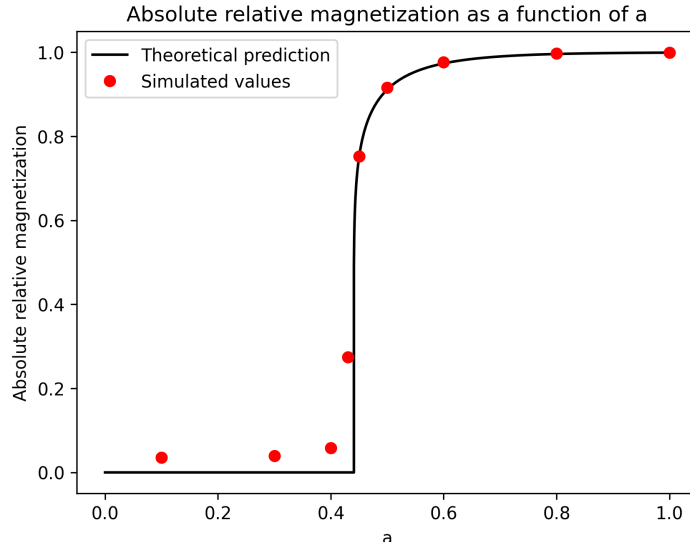


Figure 7: The absolute relative magnetisation of a 128×128 lattice as a function of a using the Wolff algorithm. The positive branch of the Onsager solution is shown for comparison. To produce the data for this plot, an initially aligned lattice was evolved for 200 iterations of the Wolff algorithm at each of 9 values of a . We take the absolute value because on a given run, the lattice randomly aligns preferentially either up or down.

mentational inconvenience associated with more sophisticated methods. For instance, a common method is to integrate the autocorrelation function, but this runs into problems because at very large times, the correlation is no longer well-described by an exponential decay. A demonstrative example of the method is shown in Fig 8.

Measurements of τ_c for each algorithm were made for a range of values of a between 0 and 1. The results are shown in Fig 9.

3.4.2 Runtime and complexity

Autocorrelation time is not the whole picture. For one thing, the Wolff algorithm is not vectorized in our implementation, which we might expect to significantly affect how the runtime compares to Metropolis-Hastings. Here we use the `time.time()` function in Python to measure the runtime of each algorithm as a function of lattice size N^2 at a fixed value of $a = 0.44$. This value is chosen because it is close to the critical point, which is where we are most interested in applying the Wolff algorithm. The results are shown in Fig 10.

3.5 Discussion

Consider the autocorrelation results. We see that the Metropolis-Hastings τ_c varies with the shape we would expect. It does not actually diverge at the critical point due to the finite size of the lattice. We propose this as also the reason why the peak value is displaced from the Onsager critical point, which is only valid for an infinite lattice. The extreme growth of τ_c for the Wolff algorithm at small a is because, as the lattice grows more disordered, the cluster size tends to one spin, meaning it takes many iterations to significantly alter the configuration. In contrast, the Metropolis-Hastings algorithm acts on the whole lattice at every iteration. This should be kept in mind when making a

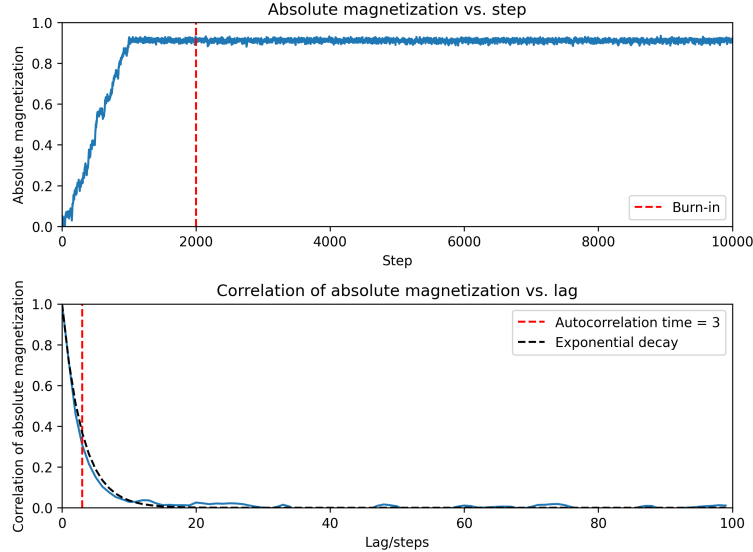


Figure 8: (Top) The absolute relative magnetization of a 128×128 initially random lattice as a function of time using the Metropolis-Hastings algorithm with $a = 0.5$. The dashed line shows the so-called ‘burn-in’, which is the time before which we discard in calculating the correlation. (Bottom) The correlation function calculated according to (4) is shown in blue. The autocorrelation time is taken to be the time at which the correlation falls to $1/e$ of its initial value. The black line shows the exponential ‘fit’ to the correlation function, and the red line shows the value of τ_c .

comparison, and it might be of interest to scale the autocorrelation times by the average number of spins interacted with per iteration.

As for the runtime results, the scaling behaviour of $O(N^2)$ for both algorithms is not surprising, since the number of spins is N^2 and, close to the critical point, the number of spins flipped per iteration by both algorithms is directly related to the system size. More surprising is the fact that the Wolff algorithm seems to initially get faster with increasing N , for which we have no explanation. It is also interesting that the Wolff algorithm is generally the faster of the two despite not being vectorized. This might be due to considering fewer spins per iteration, and is a specific. Indeed, because of how the average cluster size varies with a , we might expect the scaling relation to change entirely for other values.

Overall, we conclude that the Metropolis-Hastings algorithm performs better for simulations of the supercritical ($a < a_c$) ferromagnet and antiferromagnet due to the enormous autocorrelation time of the Wolff algorithm at small a . However, the Wolff algorithm is the better choice near the critical point and in the subcritical regime ($a > a_c$) due to the absence of critical slowing-down and the improved runtime, respectively. A full picture requires an investigation of τ_c and the complexity exploring the $a - N$ parameter space.

4 The hard-disk model

The hard-disk model is a very simple model of a 2D fluid, where the particles are modelled as non-overlapping disks of unit on a 2D torus of dimension L $[0, L)$. We are interested in computing spacial configurations of the disks.

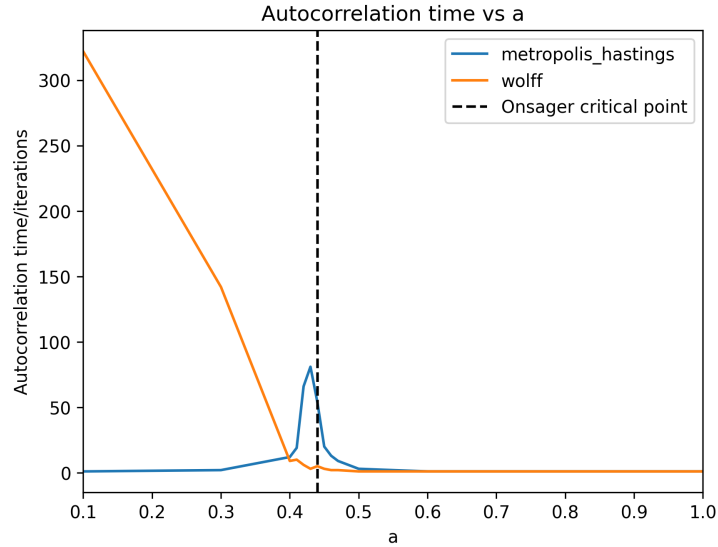


Figure 9: The autocorrelation time of the absolute relative magnetization as a function of a for the Metropolis-Hastings and Wolff algorithms. The data was produced by running each algorithm for 2000 iterations at each of 14 values of a , and then calculating the autocorrelation time using a simple $1/e$ method. We see some key features: both algorithms exhibit similar autocorrelation times at large a ; the presence of critical slowing-down is evident for the Metropolis-Hastings algorithm but not the Wolff algorithm; the Wolff algorithm exhibits extreme growth of τ_c at small a , whereas the Metropolis-Hastings algorithm sees the autocorrelation time tend back to zero. Note that we should take parts of the plot with a grain of salt, since our method really only gives an order of magnitude estimate of the autocorrelation time. We cannot necessarily take $\tau_c = 0$ to mean there is no correlation between states of the system, just that the correlation is sufficiently short-lived that we cannot detect it.

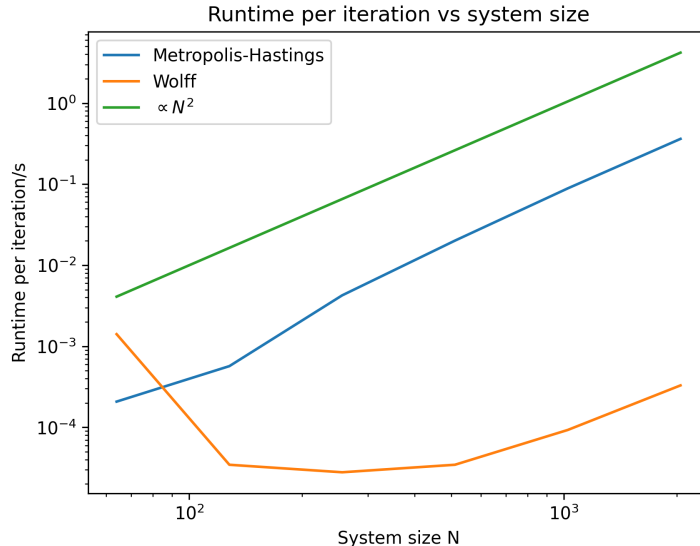


Figure 10: The runtime of the Metropolis-Hastings and Wolff algorithms per iteration as a function of lattice size N at $a = 0.44$. The data was produced by running each algorithm for 200 iterations at each of 6 values of N . The data are plotted logarithmically to show the scaling behaviour, which seems to be $O(N^2)$ for both algorithms in the limit of large N . There are some curious features present. For example the Wolff algorithm is actually faster than the Metropolis-Hastings across almost the whole range of N . Also, the Wolff algorithm actually seems to initially get faster with increasing N , which is an unusual result.

With this framing, the probability distribution we wish to sample is

$$\pi(x) = \frac{1}{Z} \mathbf{1}_{\text{no overlaps}}. \quad (8)$$

The results from Section 2 generalise to continuous state spaces [4].

4.1 Metropolis-Hastings algorithm for the hard-disk model

The Metropolis-Hastings algorithm for the hard-disk model is extremely simple:

1. Choose a disk uniformly at random.
2. If the position of the disk is (x, y) , propose a new position from a reflection-symmetric distribution $q(x, y)$
3. If the move results in an overlap, reject it. Otherwise, accept the move.

We can see that this will give (8) since

$$A(x \rightarrow y) = \min \left(1, \frac{\pi(y)q(y, x)}{\pi(x)q(x, y)} \right) = \mathbf{1}_{\text{no overlaps}},$$

where the RHS follows from the reflection symmetry of $q(x, y)$.

We can anticipate where this algorithm will run into trouble: as the number density, n , of disks increases, the acceptance ratio will tend to zero as an increasing fraction of proposed moves result in overlaps.

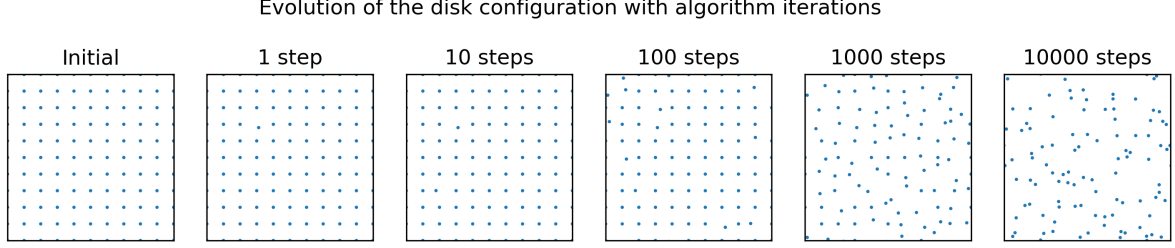


Figure 11: Evolution of an instance of the disk model with $N = 100, L = 100$ using the Metropolis-Hastings algorithm. The instance is initialized with a square lattice arrangement. The configuration is shown after 0, 1, 10, 100, 1000, 10000 iterations of the algorithm.

4.1.1 Implementation

Our Python implementation is a straightforward reproduction of the above with a few specifications: firstly, we set $q(x, y)$ to be a normal distribution centred on the disk, with its width δ being a parameter we control. The value of δ turns out to be quite important: if it is too small, the state barely evolves even at large times, but if it is too large, the acceptance ratio is very low. For all applications in this paper, setting $\delta = 1$ seemed to work. Secondly, we generate initial configurations by filling a square lattice up to N disks. Finally, we split the torus into unit squares to efficiently track overlaps. Pseudocode for the implementation is as follows:

```

occupation = array(shape=(L,L), entries=(i WHERE i is the index of
                                           the disk whose centre lies in the
                                           box,-1 IF empty))

FOR i IN iterations:
    disk = randint_uniform(0,N)
    x, y = configuration[disk]
    x_new = x + delta * random_normal(mean=0,std=1)
    y_new = y + delta * random_normal(mean=0,std=1)
    overlaps = array(entries=i WHERE i is the index whose box in
                      occupation is within 2 boxes of the box of
                      (x_new,y_new) AND
                      distance(configuration[i],(x_new, y_new)) < 2)
    IF len(overlaps) == 0:
        occupation = update_occupation(x,y,x_new,y_new,
                                       occupation)
        disks[disk] = (x_new,y_new)

```

4.1.2 Preliminary testing

As for the Ising model, we perform preliminary tests to investigate how the algorithm behaves. We initialize an instance with $N = 100, L = 100$ and evolve it with the Metropolis-Hastings algorithm, taking samples of the configuration at $t = 0, 1, 10, 100, 1000, 10000$ iterations. The results are shown in Fig 11.

We can see that the system evolves into the kind of disordered state we would expect, but it takes a large number of steps to do so.

4.2 Cluster algorithm for the hard-disk model

There are also multiple available cluster algorithms for the disk model. The most famous is the *geometric cluster algorithm* [6]. However, this is somewhat inconvenient to implement; we instead opt for a Wolff-inspired algorithm outlined in [7]. It does not seem to have a name; here we refer to it as ‘*Wolff-esque*’ or simply ‘the cluster algorithm’. It is as follows:

1. Choose a point on the torus uniformly at random. This will be the ‘pivot’.
2. Choose a disk uniformly at random to be the first member of the cluster.
3. Reflect the position of the disk in the pivot.
4. If the new position of the disk overlaps with any other disk, add each of these disks to the cluster.
5. Repeat the above two steps until all disks in the cluster have been moved.

This can be seen to satisfy detailed balance in the same way as the Metropolis-Hastings algorithm (see Section 4.1). One issue the cluster algorithm runs into is that at very high n , the cluster tends to grow to the size of the whole ensemble, at which point we actually lose the property of ergodicity [4].

4.2.1 Implementation

Once again the implementation is a straightforward reproduction, and the only specification is that we again use an occupation array to manage the overlaps. The pseudocode is as follows:

```

occupation = array(shape=(L,L), entries=(i WHERE i is the index of
                                           the disk whose centre lies in the
                                           box,-1 IF empty))

FOR i IN iterations:
    pivot = [randint_uniform(0,L),randint_uniform(0,L)]
    disk = randint_uniform(0,N)

    movers = [disk]
    WHILE len(movers) > 0:
        mover = movers.pop() # access and remove last element
        configuration[mover] = reflect(configuration[mover],pivot)
        overlaps = array(entries=i WHERE i is the index whose box in
                           occupation is within 2 boxes of the box of
                           configuration[mover] AND
                           distance(configuration[i],
                                   configuration[mover]) < 2)
        movers.append_left(overlaps)

```

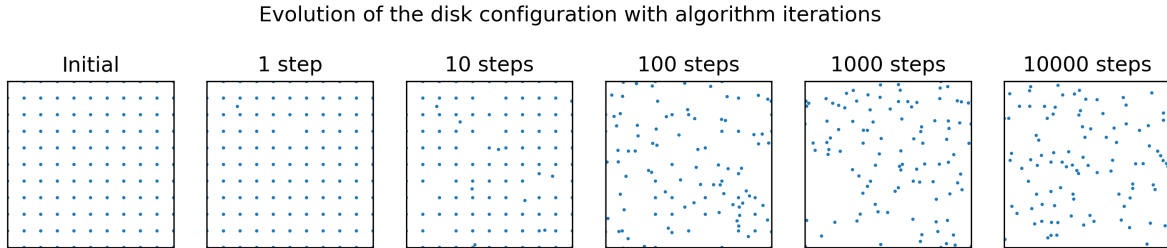


Figure 12: Evolution of an instance of the disk model with $N = 100, L = 100$ using the Wolff-esque algorithm. The instance is initialized with a square lattice arrangement. The configuration is shown after 0, 1, 10, 100, 1000, 10000 iterations of the algorithm.

4.2.2 Preliminary testing

We repeat the test described in Section 4.1 for the cluster algorithm. The results are shown in Fig 12. We see that the algorithm works as intended, and note that the system attains a visibly disordered state multiple orders of magnitude faster than the Metropolis-Hastings algorithm (compare with Fig 11).

4.3 Comparison

4.3.1 Autocorrelation time

It is clear from our preliminary tests that the cluster algorithm achieves a disordered state (which is uncorrelated with the initial state) much more quickly than the Metropolis-Hastings algorithm. Unsatisfied with only a qualitative analysis, we wish to find some macroscopic variable whose autocorrelation time we can measure².

Our approach takes inspiration from a paper which implements generalizations of the Metropolis-Hastings and Wolff-esque algorithms with a pairwise potential [8]. The former really is a trivial extension, but the latter is more subtle; it is included in Appendix B. The code for the implementations are simple modifications of the code already given, and are not included.

Using a pairwise potential, we can compute a total energy for the system, with an associated autocorrelation time. We perform autocorrelation time measurements using the same method as described in Section 3, and plot τ_c as a function of n for each algorithm in Fig 13.

4.3.2 Runtime and complexity

Again it is insufficient to consider only the number of iterations required for a good simulation. We must also consider the actual runtime and its scaling with N . We measure the runtime of each algorithm as a function of N at a fixed value of $L = 25$ using same method described in Section 3. The results are shown in Fig 14.

²Different macroscopic variables will in general have different autocorrelation times, and it is not guaranteed that comparing the autocorrelation times for one variable tells us anything about another. In this case we will hope that our chosen variable acts as a proxy the order parameter which is at least good enough to make a basic quantitative comparison. It does not seem too unreasonable to expect this, but it is not strictly justified.

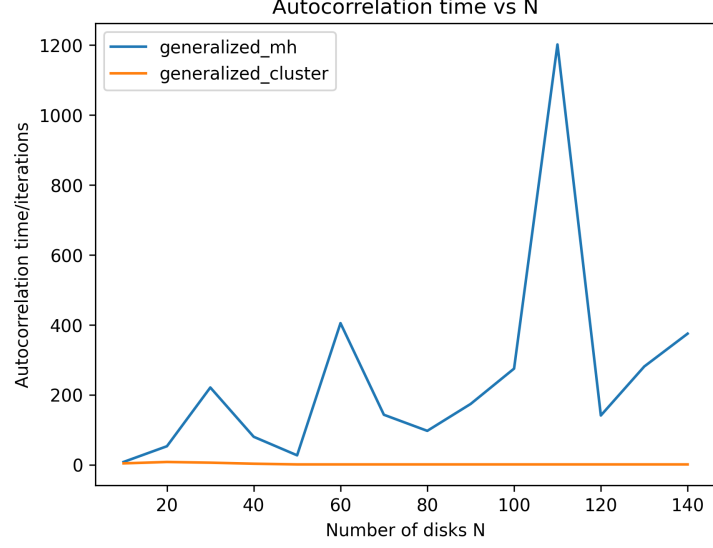


Figure 13: The autocorrelation time of the total energy as a function of the disk number N at constant $L = 25$ for the Metropolis-Hastings and Wolff-esque algorithms. The data was produced by running each algorithm for 2000 iterations at each of 14 values of N , and then calculating the autocorrelation time using a simple $1/e$ method. There is much to note about the results: firstly, we can see that the Wolff-esque algorithm has a much smaller τ_c , as expected. Secondly, and less expected, the Metropolis-Hastings algorithm seems to exhibit bizarre oscillatory behaviour. Lastly, we have essentially no detectable autocorrelation for the Wolff-esque algorithm, even at large N , although this could be due to the scale of the plot.

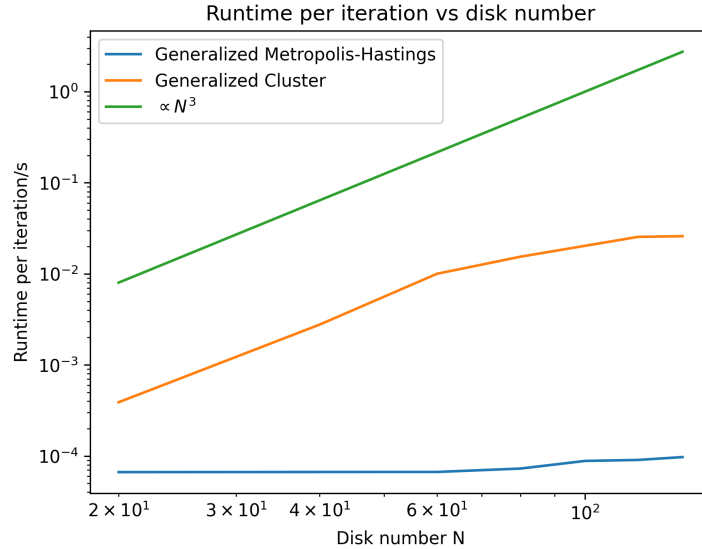


Figure 14: The runtime of the generalized Metropolis-Hastings and generalized Wolff-esque algorithms per iteration as a function of disk number N at $L = 25$. The data was produced by running each algorithm for 200 iterations at each of 7 values of N . The data are plotted logarithmically to show the scaling behaviour. We can see that the generalized cluster algorithm initially scales like N^3 before saturating to what appears to be a constant value. The generalized Metropolis-Hastings algorithm appears to have an almost constant runtime per iteration.

4.4 Discussion

The results of the autocorrelation time measurements are striking. We have no explanation for the oscillatory behaviour of Metropolis-Hastings, and frankly it calls into question the validity of the results. However, we also see in the data what we were expecting: Wolff-esque has a significantly smaller τ_c , and copes with dense systems many orders of magnitude better than Metropolis-Hastings. The caveat is that Metropolis-Hastings only interacts with a single disk per iteration, whereas Wolff-esque can interact with many disks at once, so again it may be interesting to scale the autocorrelation times by the average number of disks interacted with per iteration.

The runtime results are also interesting. We cannot directly explain the initial N^3 scaling of the generalized cluster algorithm, but it is not surprising to see that the runtime scales steeply with N given that the density will scale both the average cluster size and the number of pairwise interactions. The fact that the runtime saturates has two possible explanations: firstly, it is possible that the mechanisms underlying the complexity really do saturate at large N ; it would require a better understanding of the N^3 scaling to say for sure. Secondly, it is possible that the relatively small number of iterations is not enough to see the true scaling behaviour. This would be because the configuration would look like a square lattice for essentially the whole run, and therefore each run would look largely the same. A similar argument could be made for Metropolis-Hastings, seeing as τ_c is so large - we may not be seeing true scaling behaviour over most or all of the range of N . On the other hand, the nature of the generalized Metropolis-Hastings algorithm makes a constant runtime per iteration seem plausible, albeit perhaps not expected. We may again find scaling the results by the average number of disks interacted with per iteration instructive.

The results have many issues and unexplained features, nevertheless we can be relatively confident in the conclusion that the Metropolis-Hastings algorithm is preferable for low-density systems because of its very fast runtime, but the Wolff-esque algorithm is preferable for high- and even intermediate-density systems since, being rejection-free, it copes much better with a high probability of overlaps.

5 Conclusion

In this paper we have successfully implemented the Metropolis-Hastings algorithm, as well as cluster algorithms for the Ising model and the hard-disk model. We investigated the performance of each algorithm in terms of autocorrelation time and runtime/complexity and found the following:

- The Metropolis-Hastings algorithm for the Ising model is preferable for supercritical and antiferromagnetic systems, while the Wolff algorithm is preferable for near-critical and subcritical systems.
- The Wolff algorithm for the hard-disk model is preferable for high- and intermediate-density systems, while the Metropolis-Hastings algorithm is preferable for low-density systems.

These results come with numerous caveats, including various unexplained features of the data and unexplored questions. These are examined in detail in the discussions of Sections 3 and 4. We recommend these as the basis for future work.

References

- [1] Oskar Perron. “Zur Theorie der matrices”. In: *Mathematische Annalen* 64.2 (June 1907), pp. 248–263. DOI: 10.1007/bf01449896.
- [2] M. E. J. (Mark E. J.) Newman. *Monte Carlo methods in statistical physics / M. E. J. Newman and G. T. Barkema*. eng. 2001. ISBN: 0-19-158986-1.
- [3] Nicholas Metropolis et al. “Equation of state calculations by fast computing machines”. In: *The journal of chemical physics* 21.6 (1953), pp. 1087–1092.
- [4] Timothy Budd. *Welcome to Monte Carlo Techniques*. URL: <https://hef.ru.nl/~tbudd/mct/intro.html#>.
- [5] Ulli Wolff. “Collective Monte Carlo Updating for Spin Systems”. In: *Phys. Rev. Lett.* 62 (4 Jan. 1989), pp. 361–364. DOI: 10.1103/PhysRevLett.62.361. URL: <https://link.aps.org/doi/10.1103/PhysRevLett.62.361>.
- [6] Christophe Dress and Werner Krauth. “Cluster algorithm for hard spheres and related systems”. In: *Journal of Physics A: Mathematical and General* 28.23 (1995), p. L597.
- [7] Erik Luijten. “Introduction to Cluster Monte Carlo Algorithms”. In: vol. 703. Mar. 2007, pp. 13–38. ISBN: 978-3-540-35270-9. DOI: 10.1007/3-540-35273-2_1.
- [8] Jiwen Liu and Erik Luijten. “Generalized geometric cluster algorithm for fluid simulation”. In: *Physical Review E* 71.6 (2005), p. 066701.

A Detailed balance for the Wolff algorithm

Consider the probability of constructing an arbitrary cluster $\mathcal{C}$ (see Fig 15). Evidently, the probability of adding the same spins for the reverse move is identical: simply the probability of choosing a seed within $\mathcal{C}$ and then adding each spin with probability P_{add} . However, suppose that $\mathcal{C}$ has m neighbours with value σ and n neighbours with value $-\sigma$. Considering the probability of *not* adding those neighbours that are eligible, we see that

$$\frac{P(x \rightarrow y)}{P(y \rightarrow x)} = (1 - P_{add})^{m-n}.$$

The change in energy between the two states is $2J(m - n)$, so that the desired

$$\frac{\pi(y)}{\pi(x)} = e^{-\beta(2J(m-n))} = e^{-2a(m-n)},$$

Therefore to satisfy detailed balance (1), we must have $P_{add} = 1 - e^{-2a}$.

B Generalized cluster algorithm for the hard-disk model

The generalized cluster algorithm for the hard-disk model is as follows:

1. Choose a point on the torus uniformly at random. This will be the ‘pivot’.
2. Choose a disk i uniformly at random to be the first member of the cluster.

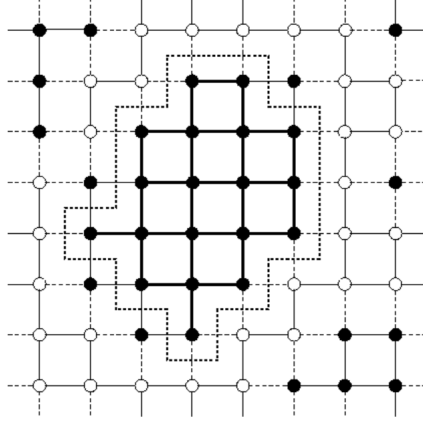


Figure 15: Schematic of a generic cluster as constructed by the Wolff algorithm. The shaded circles represent spins with value σ , while the unshaded circles represent spins with value $-\sigma$. The cluster boundary is denoted by the dashed line. Image source: <http://latt.if.usp.br/technical-pages/twawesab/Text.html/node1.html>.

3. Reflect the position of the disk in the pivot. The disk at this position is referred to as i' .
4. Each particle j that interacts with i or i' is now considered for addition to the cluster. A particle that interacts with i both in its old and in its new position is nevertheless treated once. Unlike the first particle, particle j is point-reflected with respect to the pivot only with a probability $p_{ij} = \max[1 - \exp(-\beta\Delta_{ij}), 0]$, where $\Delta_{ij} = V(|r'_i - r_j|) - V(|r_i - r_j|)$.
5. Each particle j that is indeed added to the cluster (i.e., moved) is placed on the stack. Once all particles interacting with i or i' have been considered, a particle is retrieved from the stack and all its neighbours that are not yet part of the cluster are considered in turn for inclusion in the cluster as well, following step 4.
6. Repeat the above two steps until the stack is empty.

The proof of detailed balance for this algorithm is involved, and we refer the reader to [8].

We are free to choose whatever pairwise potential we like; here we will use

$$V(r) = \begin{cases} \infty, & r < 1 \\ e^{-r}/r, & 1 \leq r < 3 \\ 0, & r \geq 3 \end{cases}$$

where the cutoff is chosen to be $r = 3$ for computational convenience.

C Code structure

The primary directory contains the following files and folders:

- Disk Model: folder containing files pertaining specifically to the the hard-disk model. It contains:

- `dm_class.py`: contains the definition of the `DiskModel` class, including methods to perform iterations of the (generalized) Metropolis-Hastings and (generalised) cluster algorithms, calculate total energy and so on.
- `disk_test.py`: contains the code used to perform the preliminary tests for each algorithm and save the resulting data.
- `disk_physics_test_plots.py`: contains the code to load the data generated by `disk_test.py` and produce and save the plots.
- `dm_complexity.py`: contains the code used to measure the runtime of each algorithm as a function of disk number N and save the resulting data.
- `dm_complexity_plot.py`: contains the code to load the data generated by `dm_complexity.py` and produce and save the plot.
- `dm_correlation_comparison.py`: contains the code used to generate and save the data for the plot of the autocorrelation times of each algorithm
- `dm_corrcomp_plot.py`: contains the code to load the data generated by `dm_correlation_comparison.py` and produce and save the plot.
- `Data`: folder containing the data generated by `disk_test.py`, `dm_complexity.py` and `dm_correlation_comparison.py`.
- `Figures`: folder containing the figures generated by `disk_physics_test_plots.py`, `dm_complexity_plot.py` and `dm_corrcomp_plot.py`.
- **General Figures**: folder containing any code to generate miscellaneous figures, along with the figures themselves. It contains:
 - `bimodal.py`: contains the code to generate the bimodal distribution plot.
 - `bimodal.png`: the plot of the bimodal distribution.
- **Ising**: folder containing files pertaining specifically to the Ising model. It contains:
 - `ising_class.py`: contains the definition of the `Ising` class, including methods to perform iterations of the Metropolis-Hastings and Wolff algorithms, calculate the magnetisation and so on.
 - `ising_test.py`: contains the code used to perform the preliminary tests for each algorithm and save the resulting data.
 - `ising_physics_test_plots.py`: contains the code to load the data generated by `ising_test.py` and produce and save the plots.
 - `magnetization_autocorrelation.py`: contains the code used to produce the plot demonstrating the autocorrelation time calculation.
 - `critical_slowing_down.py`: contains the code used to generate and save the data for the plot of the autocorrelation times of each algorithm as a function of a .
 - `csd_plot.py`: contains the code to load the data generated by `critical_slowing_down.py` and produce and save the plot.
 - `ising_complexity.py`: contains the code used to measure the runtime of each algorithm as a function of lattice size N^2 and save the resulting data.
 - `ising_complexity_plot.py`: contains the code to load the data generated by `ising_complexity.py` and produce and save the plot.

- Data: folder containing the data generated by `ising_test.py`, `ising_complexity.py` and `critical_slowing_down.py`.
- Figures: folder containing the figures generated by `ising_physics_test_plots.py`, `magnetization_autocorrelation.py`, `ising_complexity_plot.py` and `csd_plot.py`.
- `utils.py`: contains functions used for both the Ising and hard-disk simulations, namely those used to calculate the correlation function and autocorrelation time.